Métodos Cuantitativos

Clase 09 - Regresión lineal parte 2

Unidad II: Test de Hipótesis y Regresión Lineal

Breve repaso

- En la clase anterior definimos la **regresión lineal** como una herramienta estadística para modelar la relación entre una **variable dependiente** y **una o más variables independientes**.
- La idea central de la **regresión lineal es ajustar una línea (en el caso más simple) a un conjunto de puntos de datos** de manera que **se minimice la diferencia entre los valores observados y los valores predichos por el modelo**.

Conceptos claves

¿Qué veremos hoy?

- **Regresión lineal simple y residuos**
- Regresión múltiple en Python
- Supuestos del modelo
- Coeficientes de la regresión
- Evaluación del modelo

Regresión Lineal Simple

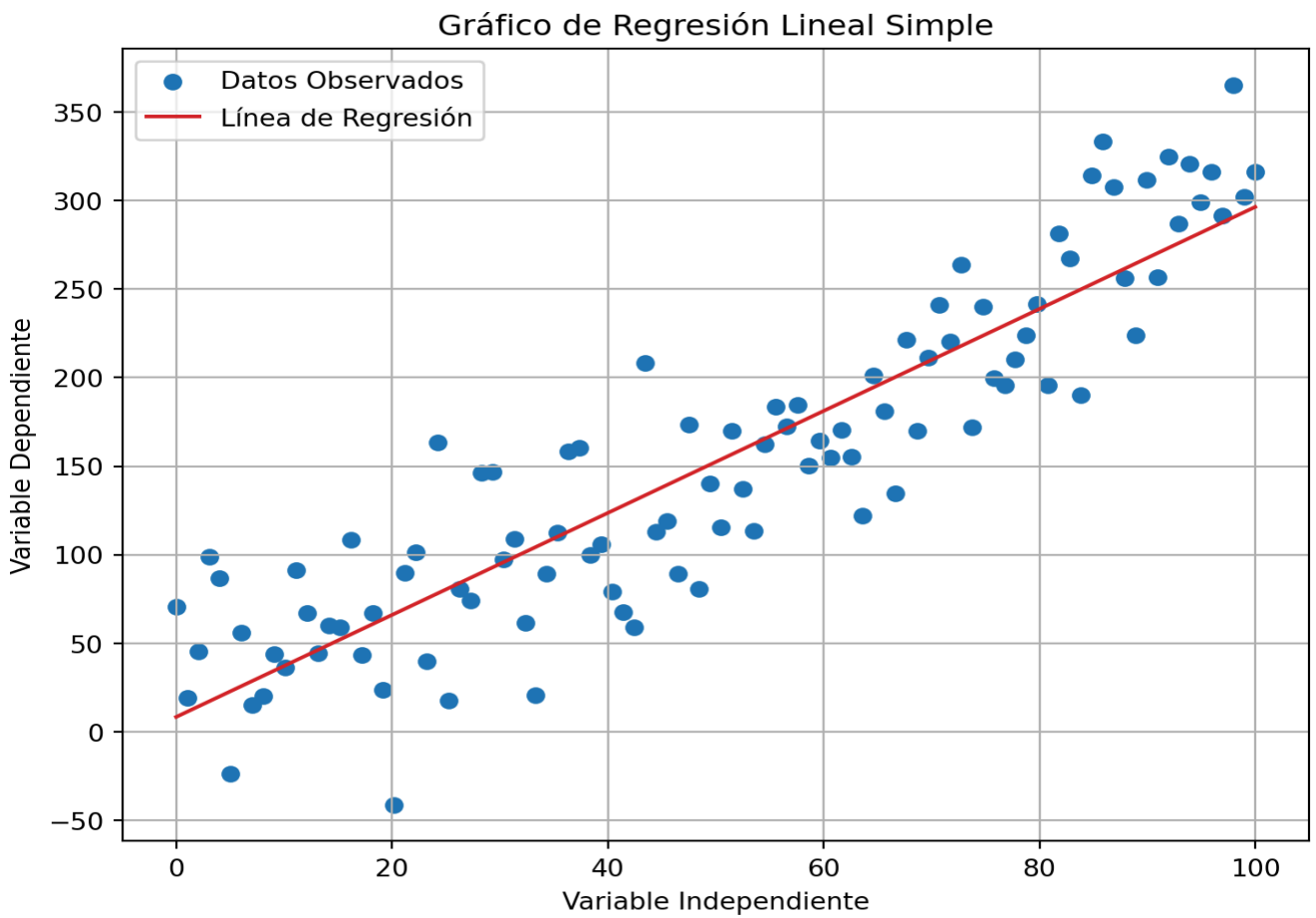
La forma general del modelo de Regresión Lineal Simple es:

$$y = \beta_0 + \beta_1 x + \varepsilon$$

Donde:

- y: Representa la variable dependiente.
- x: Representa la variable independiente.
- ε: es el término de error del modelo, que captura todas las otras influencias sobre y que no son explicadas por x.
- β_0: Es el intercepto, que representa el valor esperado de y cuando x es cero.
- β_1: Es el coeficiente de regresión para la variable independiente x, que indica cómo cambia y cuando x aumenta en una unidad.

Regresión Lineal Simple



Residuos

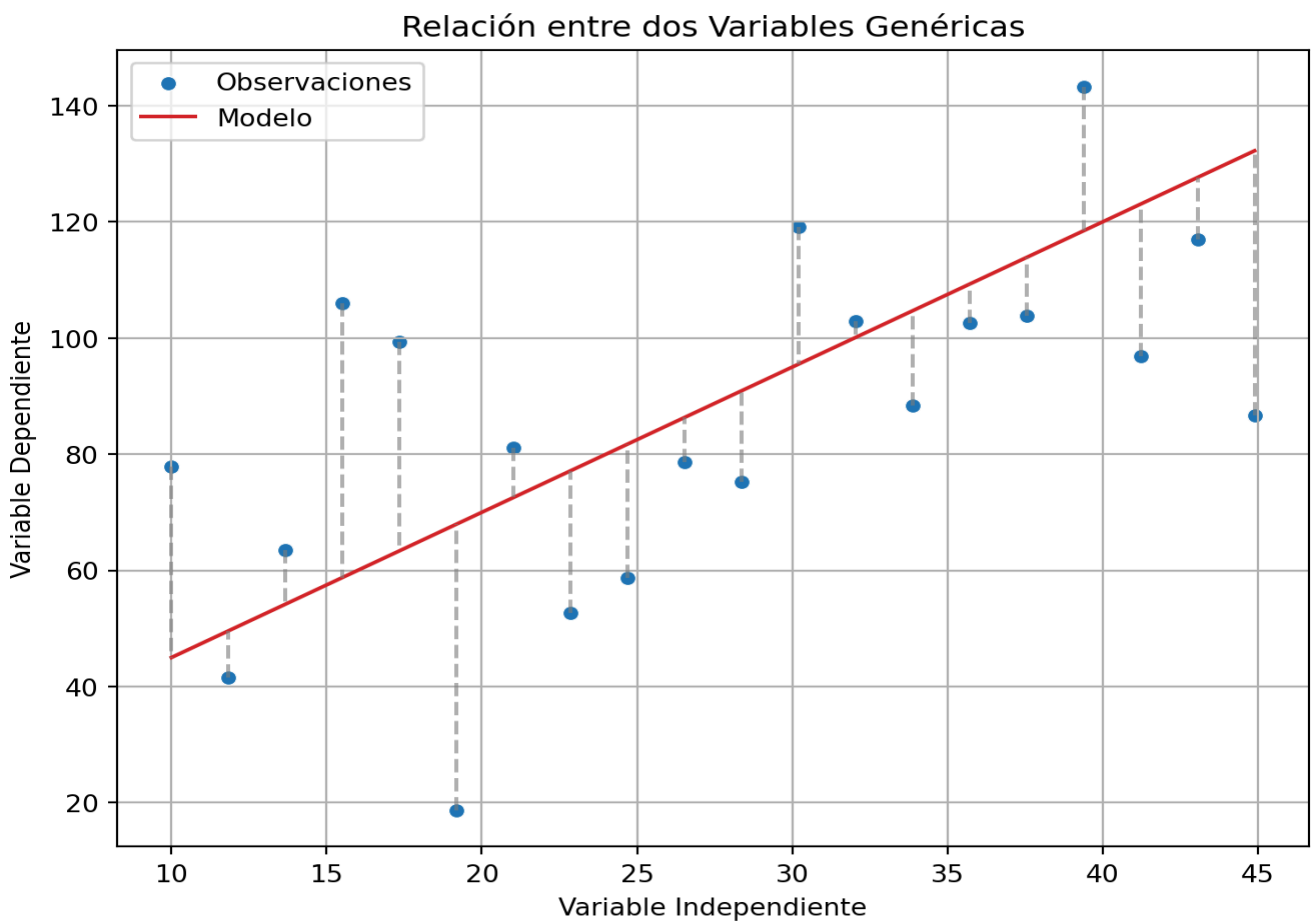
Los **residuos**, representan la discrepancia entre los valores observados y los valores predichos por nuestro modelo.

$$\varepsilon = y_i - \hat{y_i}$$

Donde:

- y_i representa el valor observado de la variable dependiente para la i-ésima observación.
- $\hat{y_i}$ representa el valor predicho por el modelo para la i-ésima observación.

Residuos


Relación entre dos Variables Genéricas
Observaciones
Modelo
Variable Dependiente
Variable Independiente
140
120
100
80
60
40
20
10
15
20
25
30
35
40
45

Error Cuadrático Medio (MSE)

- Una forma de cuantificar los errores anteriormente observados es mediante el **Error Cuadrático Medio** (MSE, por sus siglas en inglés). Esta medida se expresa de la siguiente manera:
- $\mathrm{MSE} = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y_i})^2$
- El MSE calcula el promedio de los cuadrados de los errores, es decir, la diferencia cuadrática promedio entre los valores observados y_i y los valores predichos $\hat{y_i}$.

Error Cuadrático Medio (MSE)

Supongamos que desarrollamos un modelo de regresión para calcular la temperatura en cierto mes del año:

	Día	Temp. Obs.	Predicción	Residuo
0	1.0	24.910131	22.572914	2.337216
1	2.0	22.000393	23.006309	-1.005916
2	3.0	23.946845	23.439704	0.507141
3	4.0	27.602233	23.873099	3.729134
4	5.0	27.168895	24.306494	2.862401

- Si calculamos el MSE, nos da un valor aproximado de **6.98**.
- Esto indica que, en promedio, el cuadrado de la desviación de las predicciones del modelo respecto a las observaciones reales es de aproximadamente **6.98 unidades cuadradas de temperatura**.

¿Qué veremos hoy?

- Regresión lineal simple y residuos
- **Regresión múltiple en Python**
- Supuestos del modelo
- Coeficientes de la regresión
- Evaluación del modelo

Regresión Lineal Múltiple

- La **Regresión Lineal Múltiple**, es una extensión de la regresión lineal simple.
- En este modelo, utilizamos **múltiples variables independientes** para **predecir la variable dependiente**.
- La forma general de la regresión lineal múltiple es:
  $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_k x_k + \varepsilon$
- Donde: $x_1, x_2, \ldots, x_k$ son otras variables independientes que consideramos en el modelo.

Regresión Lineal Simple en Python

En este curso, utilizaremos la librería `statsmodels` de Python para construir nuestros modelos estadísticos.

```python
import statsmodels.api as sm

# definir las variables dependiente e independiente
X = data["variable_dependiente"] # variable independiente
y = data["variable_independiente"] # variable dependiente

# añadir constante para el intercepto
X = sm.add_constant(X)

# crear el modelo con statsmodels
modelo = sm.OLS(y, X)

# ajustar el modelo
resultado = modelo.fit()

# obtener las predicciones realizadas por el modelo
y_hat = resultado.fittedvalues
```

Regresión Lineal Múltiple en Python

```python
import statsmodels.api as sm

# definir las variables dependiente e independiente
X = data["variable_dependiente"] # variable independiente
y = data[["variable_independiente_1", "variable_independiente_2"]]

# añadir constante para el intercepto
X = sm.add_constant(X)

# crear el modelo con statsmodels
modelo = sm.OLS(y, X)

# ajustar el modelo
resultado = modelo.fit()

# obtener las predicciones realizadas por el modelo
y_hat = resultado.fittedvalues
```

¿Qué veremos hoy?

- Regresión lineal simple y residuos
- Regresión múltiple en Python
- **Supuestos del modelo**
- Coeficientes de la regresión
- Evaluación del modelo

Supuestos de la Regresión Lineal

1. **Linealidad**: La relación entre las variables independientes y la variable dependiente debe ser lineal. Si la relación es no lineal, los resultados de la regresión (lineal) pueden ser poco confiables y conducir a interpretaciones erróneas sobre la relación entre las variables.
2. **Normalidad**: Los errores de la regresión deben seguir una distribución normal.
3. **Homocedasticidad**: La varianza de los errores debe ser constante en todos los niveles de las variables predictoras.

Supuestos de la Regresión Lineal

4. **Independencia**: Los errores de la regresión no deben estar correlacionados entre sí. Si hay autocorrelación presente, puede afectar la precisión de los coeficientes y las pruebas de hipótesis, lo que lleva a conclusiones erróneas sobre la importancia de las variables predictoras.

- **¿Qué es la autocorrelación?** la presencia de autocorrelación en los residuos indica que los errores del modelo muestran cierto patrón sistemático en su distribución a lo largo del tiempo.

Comprobación de supuestos

- Para ver un ejemplo de la comprobación de supuestos, utilizaremos el conjunto de datos de **riesgo crediticio**.
- La variable dependiente es `CREDIT_SCORE` y las variables independientes `R_DEBT_INCOME` y `R_GAMBLING_INCOME`.

	R_DEBT_INCOME	R_GAMBLING_INCOME	CREDIT_SCORE
0	16.0000	0.0395	444
1	4.0909	0.0000	625
2	17.3000	0.0388	469

Comprobación de supuestos

Iniciamos el entrenamiento del modelo; Ya que contamos con dos variables independientes, necesitamos ajustar un modelo de regresión lineal múltiple ($y = \beta_0 + \beta_1 x_1 + \beta_2 x_2$).

```
import statsmodels.api as sm

X = data[["R_DEBT_INCOME", "R_GAMBLING_INCOME"]]
y = data["CREDIT_SCORE"]

X = sm.add_constant(X)

modelo = sm.OLS(y, X)
resultado = modelo.fit()

residuos = resultado.resid
```

Supuestos

1. **Linealidad**
2. Normalidad
3. Homocedasticidad
4. Independencia

Comprobación de Linealidad

- Una forma de comprobar la linealidad es comprobar si la <u>media de los residuos del modelo es igual, o cercana, a 0</u>.

```
import numpy as np

print(np.mean(residuos))
```

```
-9.663381206337362e-14
```

- En este caso, se cumple, con una media cercana a 0.

Supuestos

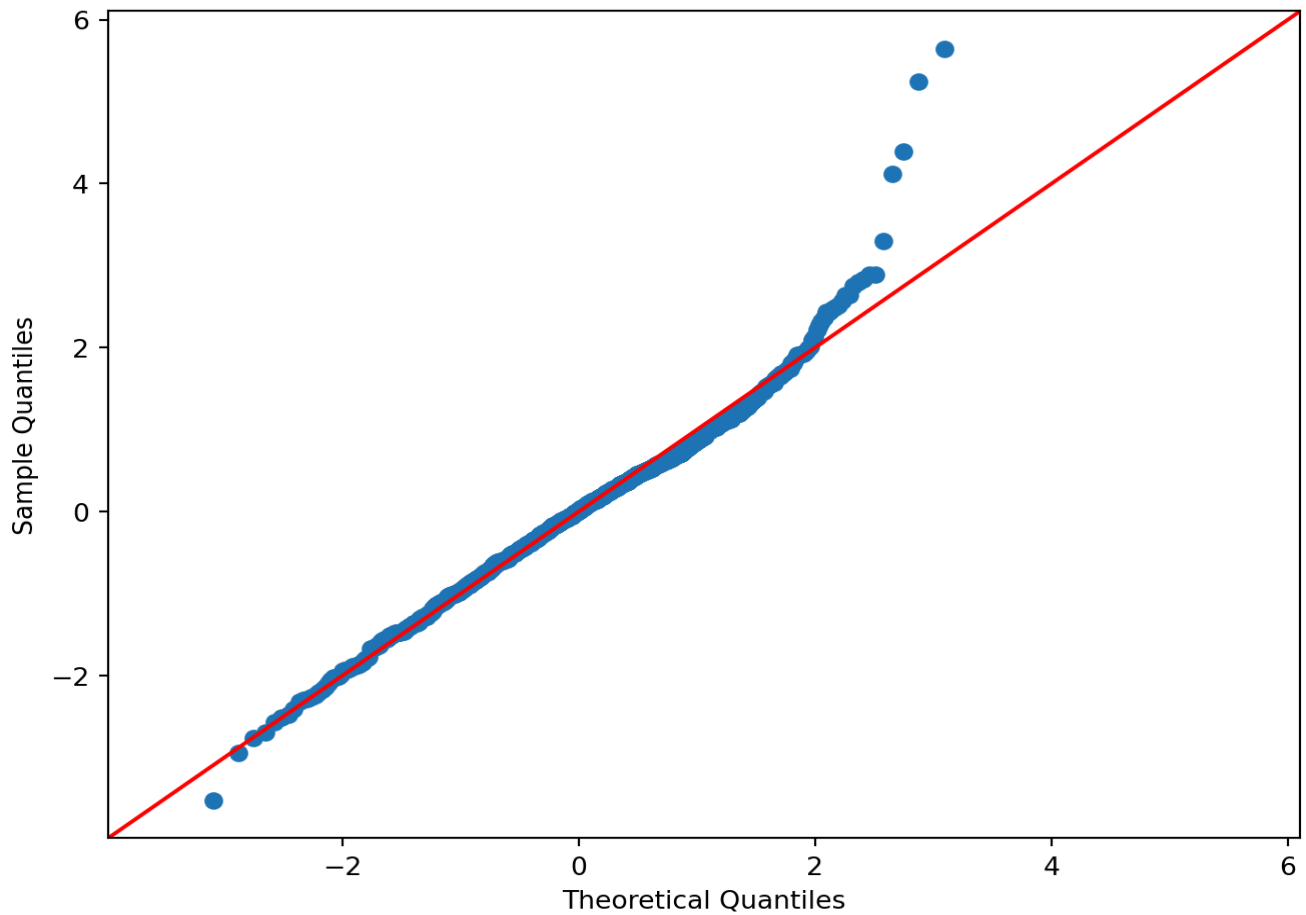
1. Linealidad
2. **Normalidad**
3. Homocedasticidad
4. Independencia

Comprobación de Normalidad

- Una forma de comprobar la normalidad de los residuos es analizar el **gráfico de Probabilidad Normal (Q-Q Plot)**.
- Este gráfico es una de las herramientas visuales más efectivas para evaluar la normalidad.
- Muestra si los cuantiles de los residuos coinciden con los cuantiles esperados de una distribución normal.
- Si los puntos en el gráfico forman aproximadamente una línea recta, esto sugiere que los residuos siguen una distribución normal.

Comprobación de Normalidad

```
sm.qqplot(residuos, line='45', fit=True)
```



Supuestos

1. Linealidad
2. Normalidad
3. **Homocedasticidad**
4. Independencia

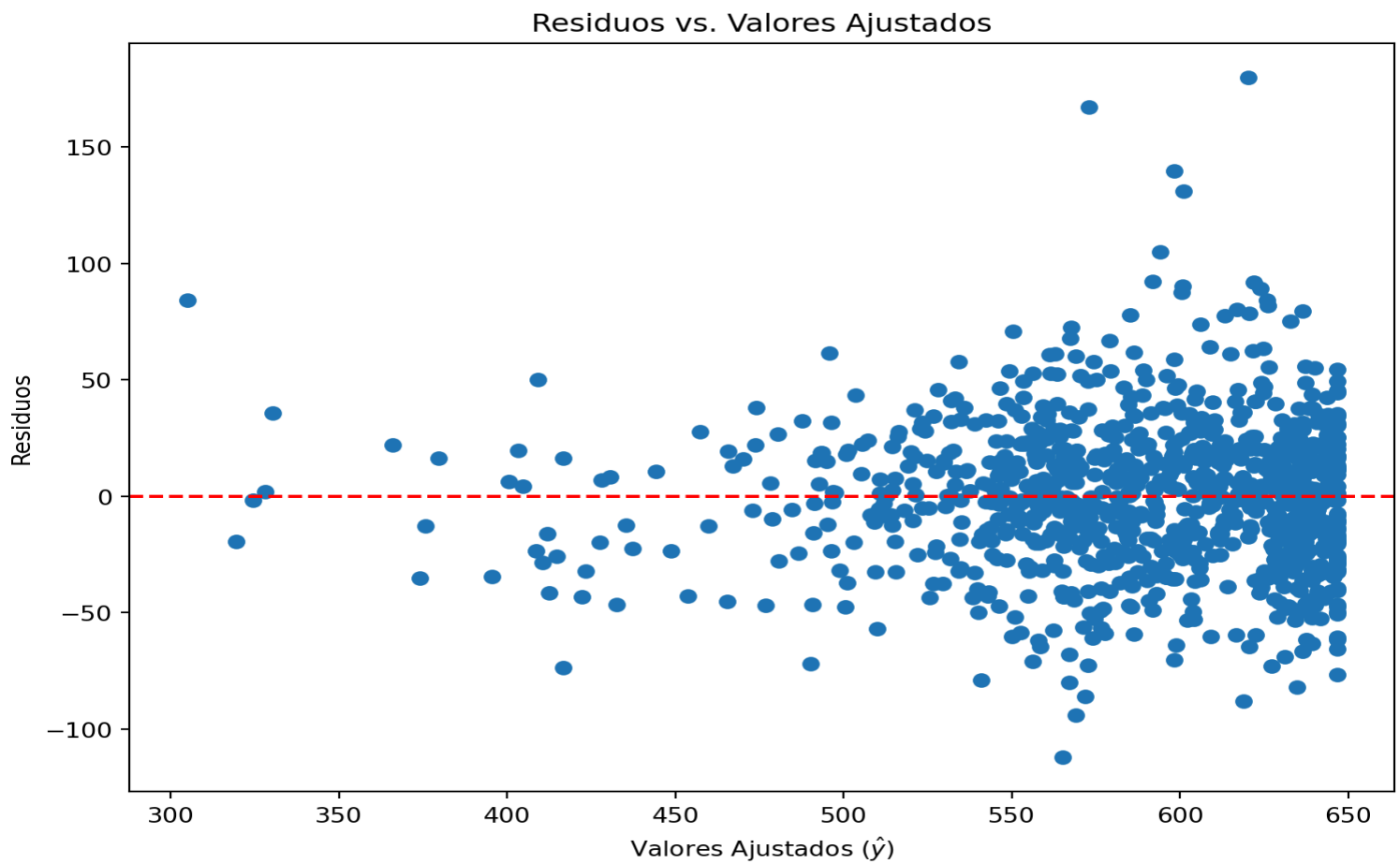
Comprobación de Homocedasticidad

- Para comprobar la homocedasticidad, que es la igualdad de varianzas en los errores de un modelo de regresión, se puede utilizar tanto métodos visuales como pruebas estadísticas.
- La homocedasticidad es importante porque asegura que el modelo es consistente y que las estimaciones de los parámetros son eficientes.

Comprobación de Homocedasticidad

- Una forma de comprobar si se cumple el supuesto de homocedasticidad es con un gráfico de dispersión de los **residuos vs valores ajustados** ($\hat{y}$).
- Lo que se busca es que los residuos presenten un patrón aleatorio alrededor de 0.
- Un patrón aleatorio sugiere que el modelo de regresión es apropiado y que los supuestos del modelo se cumplen.

Comprobación de Homocedasticidad



Comprobación de Homocedasticidad

- La prueba de Breusch-Pagan es una de las pruebas más comunes para detectar la heterocedasticidad, esta prueba plante las siguientes hipótesis:
  - H_0: La varianza de los errores es constante a lo largo de las observaciones, es decir, los errores son homocedásticos.
  - H_1: La varianza de los errores no es constante y depende de una o más variables independientes, es decir, los errores son *heterocedásticos*.

Comprobación de Homocedasticidad

Si el **valor-p de la prueba es menor que el nivel de significancia (α), se rechaza la hipótesis nula**, lo que **sugiere la presencia de heterocedasticidad**.

```python
from statsmodels.stats.diagnostic import het_breuschpagan

bp_test = het_breuschpagan(residuos, resultado.model.exog)
bp_stat, bp_pvalue, _, _ = bp_test

print(f'Breusch-Pagan Statistic: {bp_stat}, valor-p: {bp_pvalue}')
if bp_pvalue > 0.05:
    print("No existe evidencia para rechazar la hipótesis nula.")
else:
    print("Existe evidencia para rechazar la hipótesis nula, indicando pote
```

```
Breusch-Pagan Statistic: 0.5421852688013429, valor-p: 0.7625458551549342
No existe evidencia para rechazar la hipótesis nula.
```

Supuestos

1. Linealidad
2. Normalidad
3. Homocedasticidad
4. **Independencia**

Comprobación de Independencia

- La prueba de Durbin-Watson es una prueba estadística que se utiliza para analizar la presencia de autocorrelación en los residuos de un modelo de regresión.
- El estadístico de Durbin-Watson toma valores entre 0 y 4.
  - Un valor de 2 sugiere que **no hay autocorrelación**.
  - Los valores más cercanos a 0 indican **autocorrelación positiva**.
  - Los valores más cercanos a 4 indican **autocorrelación negativa**.

Comprobación de Independencia

```python
from statsmodels.stats.stattools import durbin_watson

dw_stat = durbin_watson(residuos)

print(f'Estadístico de Durbin-Watson {dw_stat}')
if dw_stat > 1.5 and dw_stat < 2.5:
    print("Los residuos parecen ser independientes.")
else:
    print("Puede haber autocorrelación en los residuos.")
```

```
Estadístico de Durbin-Watson 2.0261611304553675
Los residuos parecen ser independientes.
```

¿Qué veremos hoy?

- Regresión lineal simple y residuos
- Regresión múltiple en Python
- Supuestos del modelo
- **Coeficientes de la regresión**
- Evaluación del modelo

Coeficientes de la Regresión Lineal

Coeficientes de la Regresión Lineal

- En la regresión lineal, los coeficientes **representan la relación entre la variable dependiente y cada una de las variables independientes**.
- La interpretación de estos coeficientes es **crucial para entender el impacto que tiene cada variable independiente en la variable dependiente**.

Coeficientes de la Regresión Lineal

```python
import statsmodels.api as sm

X = data[["R_DEBT_INCOME", "R_GAMBLING_INCOME"]]
y = data["CREDIT_SCORE"]

X = sm.add_constant(X)

modelo = sm.OLS(y, X)
resultado = modelo.fit()

print(resultado.params)
```

```
const                646.578812
R_DEBT_INCOME         -9.233161
R_GAMBLING_INCOME   -207.669565
dtype: float64
```

- β_0: 646.578812
- β_1: -9.233161
- β_2: -207.669565

Coeficientes de la Regresión Lineal

- **Intercepto** (β_0): Este coeficiente representa el valor esperado de la variable dependiente cuando todas las variables independientes son cero.
- En muchos contextos, el intercepto puede no tener una interpretación práctica, especialmente si es imposible que las variables independientes tomen el valor cero simultáneamente en la realidad.

Coeficientes de la Regresión Lineal

Coeficientes de la Regresión Lineal

En el contexto de nuestro modelo:

- β_0: 646.578812
  - Este coeficiente representa el valor esperado de `CREDIT_SCORE` cuando las variables `R_DEBT_INCOME` y `R_GAMBLING_INCOME` son iguales a cero.
  - En términos prácticos, el intercepto de **646.579** indica la puntuación de crédito base, asumiendo que no hay deuda ni ingresos provenientes del juego. Este es el punto de partida de la evaluación crediticia antes de considerar el efecto de estas variables.

Coeficientes de la Regresión Lineal

- β_1: -9.233161; β_2: -207.669565
  - El coeficiente β_1 sugiere que por **cada unidad de aumento en la razón deuda/ingreso, la puntuación de crédito disminuye en 9.233 puntos**.
  - El coeficiente β_2 es considerablemente más negativo y sugiere que un aumento en la proporción de gastos de los ingresos en juegos de azar está fuertemente asociado con una disminución en la puntuación de crédito, en este caso, **una disminución de 207.67 puntos por cada unidad incrementada**.

Coeficientes de la Regresión Lineal

β_1: -9.233161; β_2: -207.669565

- **Interpretación de los Signos y Magnitudes**: Los signos negativos en ambos coeficientes indican una relación inversa con la puntuación de crédito.
  - La magnitud del coeficiente de `R_GAMBLING_INCOME` sugiere un impacto mucho más significativo en la puntuación de crédito comparado con `R_DEBT_INCOME`, lo que destaca la importancia de la fuente de ingreso en la evaluación de crédito.

¿Qué veremos hoy?

- Regresión lineal simple y residuos
- Regresión múltiple en Python
- Supuestos del modelo
- Coeficientes de la regresión
- **Evaluación del modelo**

Evaluación del modelo

Evaluación del modelo: R^2

- Una de las métricas más comunes es el **coeficiente de determinación** R^2, que nos indica la proporción de la variabilidad en la variable dependiente que es explicada por nuestro modelo.

- $R^2 = \frac{\Sigma\,(\hat{y_i} - \overline{y})^2}{\Sigma(y_i - \overline{y})^2}$
  - Un R^2 igual a 0 indica que el modelo no está capturando ninguna información útil sobre los datos.
  - Un R^2 igual a 1 indica un ajuste perfecto del modelo a los datos. Esto significa que todas las variaciones en la variable de respuesta son explicadas por el modelo.

Evaluación del modelo: R^2

```python
import statsmodels.api as sm

X = data[["R_DEBT_INCOME", "R_GAMBLING_INCOME"]]
y = data["CREDIT_SCORE"]

X = sm.add_constant(X)

modelo = sm.OLS(y, X)
resultado = modelo.fit()

print(resultado.rsquared)
```

```
0.7475184452495501
```

Evaluación del modelo: R_a^2

- Sin embargo el R^2 **puede ser engañoso en modelos con múltiples variables predictoras**, por lo que también consideraremos el R^2 ajustado (R_a^2), que penaliza la inclusión de variables irrelevantes en el modelo.

- $$R_a^2 = 1 - [(\frac{n-1}{n-k-1})] \cdot (1 - R^2)$$

- Donde:
  - R^2: coeficiente de determinación.
  - n: número de observaciones.
  - k: número de variables independientes.

Evaluación del modelo: R_a^2

```python
import statsmodels.api as sm

X = data[["R_DEBT_INCOME", "R_GAMBLING_INCOME"]]
y = data["CREDIT_SCORE"]

X = sm.add_constant(X)

modelo = sm.OLS(y, X)
resultado = modelo.fit()

print(resultado.rsquared_adj)
```

```
0.7470119626923777
```

Evaluación del modelo: RMSE

Evaluación del modelo: RMSE

```python
import statsmodels.api as sm
from statsmodels.tools.eval_measures import rmse

X = data[["R_DEBT_INCOME", "R_GAMBLING_INCOME"]]
y = data["CREDIT_SCORE"]

X = sm.add_constant(X)

modelo = sm.OLS(y, X)
resultado = modelo.fit()

y_hat = resultado.fittedvalues
print(rmse(y, y_hat))
```

```
31.8479812791391
```

- El resultado de 31.85 significa que, en promedio, **las predicciones del modelo difieren de los valores reales en aproximadamente 31.85 unidades en la escala de la variable de respuesta** (en este caso, la puntuación de crédito).

Bibliografía recomendada

- Estadística, Capítulo 10, Triola Mario F, 2018 12a edición.

Actividad práctica

- A continuación, para poner en práctica todos los conceptos aprendidos, realizaremos una actividad guiada en Colab.